Chapter 1 Test: Data Types & Feature Understanding

Level 1 Warmup + Level 2 Interviewer Question Bank

Answering Way: Descriptive, Coding, and Scenario-Based answers below are written the way a thoughtful student would explain them out loud - conversational tone, brief code where useful, and always touching on the 'why' behind the concept, not just the mechanics.

Level 1: Warmup Test (10 Questions)

Section A - Multiple Choice Questions (5)

Q1. Which type of variable has categories with a meaningful order, but the gaps between categories are not necessarily equal?

A. Nominal Categorical
B. Ordinal Categorical
C. Discrete Numerical
D. Continuous Numerical

Answer & Explanation

Correct Answer: B. Ordinal data has a real order (e.g. 'Junior' < 'Senior' < 'Lead', or a 1-5 star rating) but the distance between each step isn't guaranteed to be equal, unlike numerical data.

Q2. In pandas, the 'object' dtype most commonly represents:

A. Only numeric values that failed to parse
B. Text strings, which may be free text OR categorical labels
C. Boolean True/False values exclusively
D. Datetime values that haven't been parsed yet

Answer & Explanation

Correct Answer: B. 'object' is pandas' catch-all for text. It could be a clean nominal/ordinal category ('Male', 'Female') or messy free text (product reviews) - you always need to sample the actual values to know which.

Q3. A numerical column has only 4 unique values across 10,000 rows. What should you suspect?

A. It's definitely a continuous numerical column
B. It's likely a disguised categorical column
C. It must be a datetime column
D. It's an error and should be dropped immediately

Answer & Explanation

Correct Answer: B. Very low unique-value counts (a common heuristic is nunique() < 15) on a numeric column is a strong signal it's actually a category in disguise, like a rating scale or a coded status.

Q4. Which of the following is an example of unstructured data?

A. A SQL table with named columns and fixed types
B. A CSV file with rows and columns
C. A folder of customer product review text and photos
D. A JSON object with consistent, well-defined keys

Answer & Explanation

Correct Answer: C. Free-text reviews and images have no fixed schema or predictable columns - that's the definition of unstructured data. The other three options all have some predictable structure.

Q5. A categorical column has 500 unique values out of 1,000 rows. What's generally the best encoding approach?

A. One-hot encoding (create 500 new columns)
B. Target encoding, frequency encoding, or hashing
C. Treat it as free text and ignore it
D. Convert it to a boolean column

Answer & Explanation

Correct Answer: B. One-hot encoding at this cardinality would create 500 sparse columns - a memory and modeling nightmare. Target/frequency encoding or hashing compresses this information into one or a few columns instead.

Section B - Descriptive Questions (3)

Q6. Why is it a mistake to treat an ID column (like customer_id = 1001, 1002...) as a regular numerical feature? Walk through what could actually go wrong.

Answer

So the core issue here is that an ID is a label, not a quantity - it just happens to be written using digits. The number 1002 isn't 'more' than 1001 in any meaningful sense; it's just a different name tag.

If you leave it as int64 and feed it into a model, a few bad things can happen. First, something like mean(customer_id) is completely meaningless - averaging labels makes no sense. Second, and more dangerously, a model might pick up a spurious correlation. I saw this exact pattern in the manufacturing example from the chapter: machine_id was left as an integer, and since newer machines happened to have higher IDs and also had fewer failures (because they're newer), the model 'learned' that higher machine_id causes lower failure rate - which is nonsense, it's really just a proxy for machine age.

The fix is simple: cast ID columns to string/object right away with .astype(str), so no numeric encoder or aggregation function accidentally treats it as a quantity. If you want to keep it as a feature at all (rather than just a join key), one-hot or target encoding as a categorical is the right move.

Q7. Explain the difference between structured, semi-structured, and unstructured data, with an example of each.

Answer

I like to think of this as a spectrum of 'how predictable is the shape of this data before I even look at it'.

Structured data is the most predictable - think of a SQL table or a clean CSV. Every row has the same columns, every column has a known type, and you can query it directly. Example: a transactions table with order_id, amount, and status columns.

Unstructured data has no fixed shape at all - a product review, a doctor's handwritten note, or an image. You can't just 'load' it into rows and columns; it needs specialized processing first (NLP for text, computer vision for images) before it becomes usable.

Semi-structured sits in between - it has some organization (keys, tags) but isn't a strict table. JSON API responses are the classic example: {'order_id': 101, 'customer': {'name': 'Amit', 'city': 'Pune'}} has clear keys, but the nesting means it isn't flat like a spreadsheet. In pandas, pd.json_normalize() is exactly the tool that flattens this into a proper table.

And honestly, most real datasets are hybrid - a transactions table (structured) with a 'notes' column (unstructured free text) sitting right next to it. Part of EDA is spotting that hybrid mix early, so you don't try to one-hot encode a free-text column by accident.

Q8. When you calculate both the mean and median of a numerical column and they're very different, what does that tell you, and why does it matter for something like missing value imputation?

Answer

A big gap between mean and median is basically a red flag for skew - usually caused by one or a handful of extreme outliers pulling the mean away from where 'most' of the data actually sits.

The salary example from the chapter is a great illustration: five salaries clustered around 45k-61k, plus one at 250k. The mean comes out to about 84,833 - which doesn't represent almost anyone in the dataset - while the median (52,500) is a much more honest 'typical' value.

Why it matters for imputation specifically: if you fill missing salary values with the mean, you're injecting a value inflated by that one outlier into every missing row. That systematically biases your dataset upward. Using the median instead gives you a fill value that's actually representative of a typical row, which is why the chapter's rule of thumb is: whenever mean and median diverge noticeably, default to median-based statistics (and mention the outlier in your EDA notes rather than quietly averaging over it).

Section C - Coding Questions (2)

Q9. Given a DataFrame `df` with a mix of numeric, text, and datetime columns, write code to automatically split the column names into three lists: numerical_cols, categorical_cols, and datetime_cols.

Answer

This is really just leaning on select_dtypes(), which is the fastest first-pass way to bucket columns before doing any deeper inspection:

```
import pandas as pd

numerical_cols = df.select_dtypes(include=['number']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object',
'bool']).columns.tolist()
datetime_cols = df.select_dtypes(include=['datetime']).columns.tolist()

print('Numerical :', numerical_cols)
print('Categorical:', categorical_cols)
print('Datetime :', datetime_cols)
```

One thing worth flagging: this only tells you what pandas currently thinks the type is, not what it should be. A 'customer_id' column would land in numerical_cols here even though it's really a label - so this is step one of the workflow, not the final answer. You'd still want to sanity-check with .nunique() and .sample() afterward.

Q10. Write code that scans a DataFrame's numeric columns and flags any that have fewer than 15 unique values, since these are likely disguised categorical columns.

Answer

This is the exact 'low cardinality on a numeric column' heuristic from the chapter - it's a cheap check that catches a surprising number of real bugs:

```
import pandas as pd

num_cols = df.select_dtypes(include='number').columns
low_card = [c for c in num_cols if df[c].nunique() < 15]

print('Numerical columns that look like disguised categoricals:',
low_card)
```

If a column like 'gender' (encoded 0/1) or 'rating' (1-5) shows up in that low_card list, that's your cue to go back and re-map it with explicit labels or treat it as ordinal/categorical rather than leaving it as a plain float or int - otherwise you risk something like averaging a rating scale as if it were a real quantity.

Level 2: Interviewer Test (10 Intermediate / Semi-Advanced Questions)

Section A - Scenario-Based MCQ (3)

Q11. *[Scenario]* Scenario: You load a dataset and find a 'zip_code' column stored as int64, with 500 unique values across 1,000 rows. What's the most appropriate next step?

A. Leave it as int64 and feed it directly into a regression model

B. Cast it to string/object, then consider hashing or target encoding since it's medium/high cardinality

C. One-hot encode all 500 values immediately

D. Drop it - any column with more than 100 unique values is useless

Answer & Explanation

Correct Answer: B. Zip codes are nominal labels, not quantities, so the numeric dtype is misleading. At 500 unique values it also sits in high-cardinality territory, where one-hot would be wasteful - hashing or target encoding (or extracting a broader region) is the practical move.

Q12. *[Scenario]* Scenario: You compute mean = 4.2 lakhs and median = 1.1 lakhs for a bank's loan amounts, and the top 5% of borrowers are large corporate accounts. Which statistic should drive your reporting and imputation decisions for the retail segment?

A. Mean, because it uses all the data

B. Median, since it isn't distorted by the small number of very large corporate loans

C. Mode, since it's the simplest to compute

D. Standard deviation, since it captures spread

Answer & Explanation

Correct Answer: B. This mirrors the chapter's finance example exactly - the mean was skewed 3x by a small number of large corporate loans. Splitting by segment and using the median gives a far more honest picture of what a typical retail borrower actually looks like.

Q13. *[Scenario]* Scenario: An API returns records like {'order_id': 101, 'customer': {'name': 'Amit', 'city': 'Pune'}}. Which pandas function correctly flattens this into a table with columns like 'customer.name' and 'customer.city'?

A. pd.merge()

B. pd.json_normalize()

C. pd.pivot_table()

D. pd.concat()

Answer & Explanation

Correct Answer: B. pd.json_normalize() is purpose-built for flattening nested JSON/dict structures into a flat DataFrame, joining nested key names with a dot by default.

Section B - Scenario-Based Descriptive (2)

Q14. *[Scenario]* Scenario: Your team included 'machine_id' (an integer like 1001, 1002...) directly as a numerical feature in a regression model predicting failure rate. The model shows a strong negative relationship: higher machine_id → lower failure rate. A colleague is excited about this 'insight'. What's actually going on, and what would you do?

Answer

Before getting excited, I'd want to pump the brakes here - this smells like a classic type-mismatch-driven spurious correlation, and the chapter's manufacturing example is almost this exact scenario.

What's likely happening: machine_id was assigned sequentially as machines were installed, so higher IDs simply mean 'installed more recently'. Newer machines naturally have fewer failures because they haven't had time to wear down yet - not because a bigger ID number causes reliability. The model isn't learning anything about machine identity; it's accidentally learning a proxy for machine age.

What I'd actually do: first, cast machine_id to a categorical/string type so it's never treated as an ordered quantity: df['machine_id'] = df['machine_id'].astype(str). Then decide how to actually use it - if there are only a handful of distinct machines, one-hot encoding is fine; if there are hundreds, I'd look at target or frequency encoding instead. And separately, if 'machine age' is really the driver, I'd explicitly engineer that as its own feature (e.g. install_date to compute age) rather than letting an ID column smuggle that signal in sideways.

Q15. *[Scenario]* Scenario: You've just inherited a 140-column dataset from a data partner with zero documentation. Walk through the process you'd follow before doing any modeling, and explain why skipping this step is risky.

Answer

Honestly, this is exactly the situation the chapter's variable inventory concept is built for, so I'd follow that same systematic path rather than randomly poking at columns.

First, I'd run the cheap, high-level checks across every column at once: df.dtypes and df.info() to see what pandas currently thinks each column is, plus memory usage. Then df.nunique() across the board to catch two things immediately - constant columns (nunique == 1, zero predictive value) and disguised categoricals hiding in numeric columns.

Next, I'd do a schema inspection pass: check value ranges for anything numeric (are there negative ages, percentages over 100?), check categorical columns against sane value lists, and check df.isnull().sum() and df.duplicated().sum() for null and duplicate issues.

Then I'd actually build the variable inventory - a table with dtype, variable type, cardinality bucket, missing %, a few sample values, and a recommended treatment (keep/drop/impute/encode) for every single column. I'd generate this programmatically, not by hand in Excel, so it stays accurate if the data refreshes.

Why this matters: the chapter's own example is a great cautionary tale - two analysts spent 3 combined days manually investigating columns one by one, when a 5-minute programmatic inventory immediately flagged 12 constant columns, 8 quasi-unique ID columns, and 3 columns with over 70% missing data. Skipping this step means you risk building features on garbage columns, or worse, having every team member independently re-discover (and maybe disagree about) the same issues.

Section C - Coding Questions (3)

Q16. Write a data quality check function that flags rows with an invalid 'age' (outside 0-120) and an unexpected 'gender' value (anything other than 'M' or 'F').

Answer

This is basically automating the schema inspection checklist from the chapter - range checks plus an allowed-category check:

```
import pandas as pd

def check_quality(df):
 invalid_age = df[(df['age'] < 0) | (df['age'] > 120)]
 valid_genders = {'M', 'F'}
 invalid_gender = df[~df['gender'].isin(valid_genders)]
 print('Invalid age rows:', len(invalid_age))
 print('Invalid gender values found:',
invalid_gender['gender'].unique())
 return invalid_age, invalid_gender

invalid_age_rows, invalid_gender_rows = check_quality(df)
```

The key habit here is returning the actual offending rows, not just a count - so whoever reviews this can immediately go look at those specific records rather than guessing what's wrong. I'd usually wrap this kind of check in an assert or log-and-alert step in a production pipeline so it fails loudly rather than silently.

Q17. Write a function that takes a DataFrame and classifies every column's cardinality into Binary, Low, Medium, High, or Quasi-Unique.

Answer

This is basically packaging the cardinality decision table from the chapter into one reusable function so you're not eyeballing nunique() column by column:

```
import pandas as pd

def classify_cardinality(n_unique, n_rows):
 pct = n_unique / n_rows
 if n_unique == 1:
 return 'Constant - DROP'
 elif n_unique == 2:
 return 'Binary'
 elif n_unique <= 15:
 return 'Low'
 elif n_unique <= 100:
 return 'Medium'
 elif pct > 0.5:
 return 'Quasi-Unique'
 else:
 return 'High'

cardinality_report = pd.DataFrame({
 'unique_values': df.nunique(),
 'total_rows': len(df),
})
cardinality_report['category'] = [
 classify_cardinality(u, len(df)) for u in
cardinality_report['unique_values']
]
print(cardinality_report)
```

The nice thing about writing it this way is that 'category' now directly tells you the encoding strategy: Binary → simple 0/1, Low → one-hot, Medium/High → target/frequency encoding or hashing, Quasi-Unique → drop or extract a feature from it (like email domain).

Q18. Write code to clean a 'salary' column that contains values like '₹50,000', '₹1,20,000', and 'N/A', converting it into a proper float column.

Answer

This is a multi-step text-to-numeric cleanup - strip the symbols, handle the null placeholder, then convert safely:

```
import pandas as pd
import numpy as np

df['salary'] = (
 df['salary']
 .str.replace(r'[₹,]', '', regex=True)
 .str.strip()
 .replace('N/A', np.nan)
)
df['salary'] = pd.to_numeric(df['salary'], errors='coerce')

print(df['salary'].dtype)
print(df['salary'])
```

Two details I'd never skip here: using errors='coerce' means any value that still can't be parsed becomes NaN instead of crashing the whole pipeline, and replacing 'N/A' before the numeric conversion means it becomes a proper missing value you can impute later, rather than an error.

Section D - Database Scenario-Based Coding (2)

Q19. *[Database Scenario]* Database Scenario: You suspect a 'customers' SQL table has duplicate customer records based on email address. Write a SQL query to find every email that appears more than once, along with how many times it appears.

Answer

This is the SQL equivalent of df.duplicated() but working on a key column rather than whole rows - group by the key, then filter groups with more than one member:

```
SELECT
 email,
 COUNT(*) AS occurrence_count
FROM customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY occurrence_count DESC;
```

The HAVING clause is the important bit here - you can't filter on an aggregate like COUNT(*) with a plain WHERE, since WHERE runs before grouping happens. This query gives you exactly the key-based duplicates the chapter talks about: same email, but possibly different values in other columns, which is a more subtle problem than a fully identical row.

Q20. *[Database Scenario]* Database Scenario: Before pulling a 'patients' table into pandas, you want to check the SQL table directly for schema violations - specifically, rows where 'age' is negative or over 120, since this happened once before at the source system. Write a SQL query to surface these rows.

Answer

This is just moving the range-check part of schema inspection upstream, so you catch bad data at the source instead of after loading it into pandas:

```
SELECT *
FROM patients
WHERE age < 0 OR age > 120;
```

I like running this kind of check directly in SQL when possible, because it scales to tables far bigger than you'd want to pull into memory just to inspect. In practice I'd pair this with a quick count query too - SELECT COUNT(*) FROM patients WHERE age < 0 OR age > 120 - so an automated pipeline can assert that count is zero and raise an alert the moment it isn't, the same spirit as the chapter's assert df['stock_quantity'].min() >= 0 example.